NYC Urban Mobility Data Explorer

Technical Documentation Report

Team Name	Code Sync
Members	Alier Piel · Chol Mach Kuol · Kuol Akech · Abay Tessema
Course	Enterprise Web Development
Submission Date	Thursday, 15 October 2026

1. Problem Framing and Dataset Analysis

Dataset Description

The New York City Taxi and Limousine Commission (TLC) publishes monthly trip records for all yellow taxi rides in New York City. For this project we used the January 2019 dataset, which contains **7,667,792 raw trip records**. Each record captures the full lifecycle of a single taxi ride: pickup and dropoff timestamps, pickup and dropoff zone IDs, trip distance in miles, an itemised fare breakdown covering the base fare, extra charges, MTA tax, tip amount, tolls, improvement surcharge, congestion surcharge and total amount, plus payment type, rate code, vendor ID and passenger count.

The dataset was integrated with two supporting files. `taxi_zone_lookup.csv` maps the 265 TLC zone IDs to borough names and service zone categories and becomes the zones and boroughs dimension tables in the database. The taxi zone shapefile holds the zone polygon boundaries, which the pipeline reads and reprojects to longitude and latitude coordinates to produce `taxi_zones.geojson`. The loader stores the geometry in the zones table and the frontend uses it to draw the choropleth map.

Data Challenges

The raw dataset contained several categories of physically and logically impossible records. The pipeline processes each record against a sequence of rules and assigns it to the first rule it breaks, so every excluded record has exactly one reason:

Exclusion Reason	Records Removed
Zero distance (cancelled rides or GPS failures)	48,691
Duration over 6 hours (meter left running)	20,463
Fare zero or negative	6,567
Dropoff not after pickup (logical impossibility)	6,291
Speed over 80 mph (physically impossible in NYC)	5,921
Pickup outside January 2019 (meter clock errors)	537
Fare below $2.50 TLC flag-drop minimum	391
Distance over 100 miles (odometer or GPS error)	31
Negative charge component	6
Fare above $500 (data-entry error)	4
Total excluded	**88,902 (1.159% of raw dataset)**

Every excluded record is stored in the `excluded_records` table with its reason code, linking back to the original raw file row via `source_row`. The dashboard's data quality panel charts all exclusion categories so the cleaning decisions are fully visible. Beyond exclusions, a further **369,285 records** were kept but flagged or repaired:

Repair / Flag Reason	Records Affected
Unknown dropoff zone (TLC zone IDs 264 and 265)	154,335
Unknown pickup zone	152,317
Zero passengers — set to NULL	115,074
Undocumented vendor ID	76,380
Total amount not equal to sum of fare components	15,751
Invalid rate code — set to NULL	106

Assumptions Made During Cleaning

- Any trip with zero or negative duration was removed as logically impossible regardless of distance.
- Speed was calculated as distance divided by duration in hours. Anything above 80 mph was excluded as physically impossible on NYC roads.
- The $2.50 threshold for fare exclusion was chosen because it is the TLC-mandated flag-drop fare. No metered ride can cost less. Approximately 98% of fares below this threshold were placeholder values such as $0.01 for a 33-mile ride.
- Zero passengers was not treated as an invalid trip. It is a common driver data-entry habit. The value was stored as NULL to indicate unknown rather than deleting 115,074 otherwise valid rides.

- Trips with unknown zones (264 and 265) were kept because the trip data itself — fare, distance, duration — is valid. Only the geographic attribution is missing. The loader labels them Unknown and Outside NYC so the dashboard shows them explicitly.

Derived Features

The pipeline's `add_features` function computes nine features from the raw columns. The four analytically significant ones are:

- **trip_duration_min:** dropoff minus pickup in minutes. Basis for speed and directly measures how long rides take at each hour. Zero, negative and over-6-hour durations are already excluded before this is calculated.
- **avg_speed_mph:** distance divided by duration in hours. Measures congestion directly and catches physically impossible trips above 80 mph.
- **fare_per_mile:** fare divided by distance. Shows the cost of traffic — because the TLC meter charges for time spent slow or stopped as well as distance, a slow mile costs more. Left NULL for trips under 0.25 miles to avoid misleadingly high figures from the flag-drop minimum.
- **tip_pct:** tip divided by fare multiplied by 100. Only calculated for credit card payments with a fare of at least $2.50, because cash tips are recorded as zero regardless of what was actually paid.

The five helper features are `pickup_hour`, `pickup_dow`, `is_weekend`, `time_of_day` (night, morning rush, midday, evening rush, evening) and `is_airport_trip`, which flags any trip where either the pickup or dropoff zone is JFK, LaGuardia or Newark (zone IDs 132, 138 and 1). These allow the dashboard to filter and group without recalculating anything per row at query time.

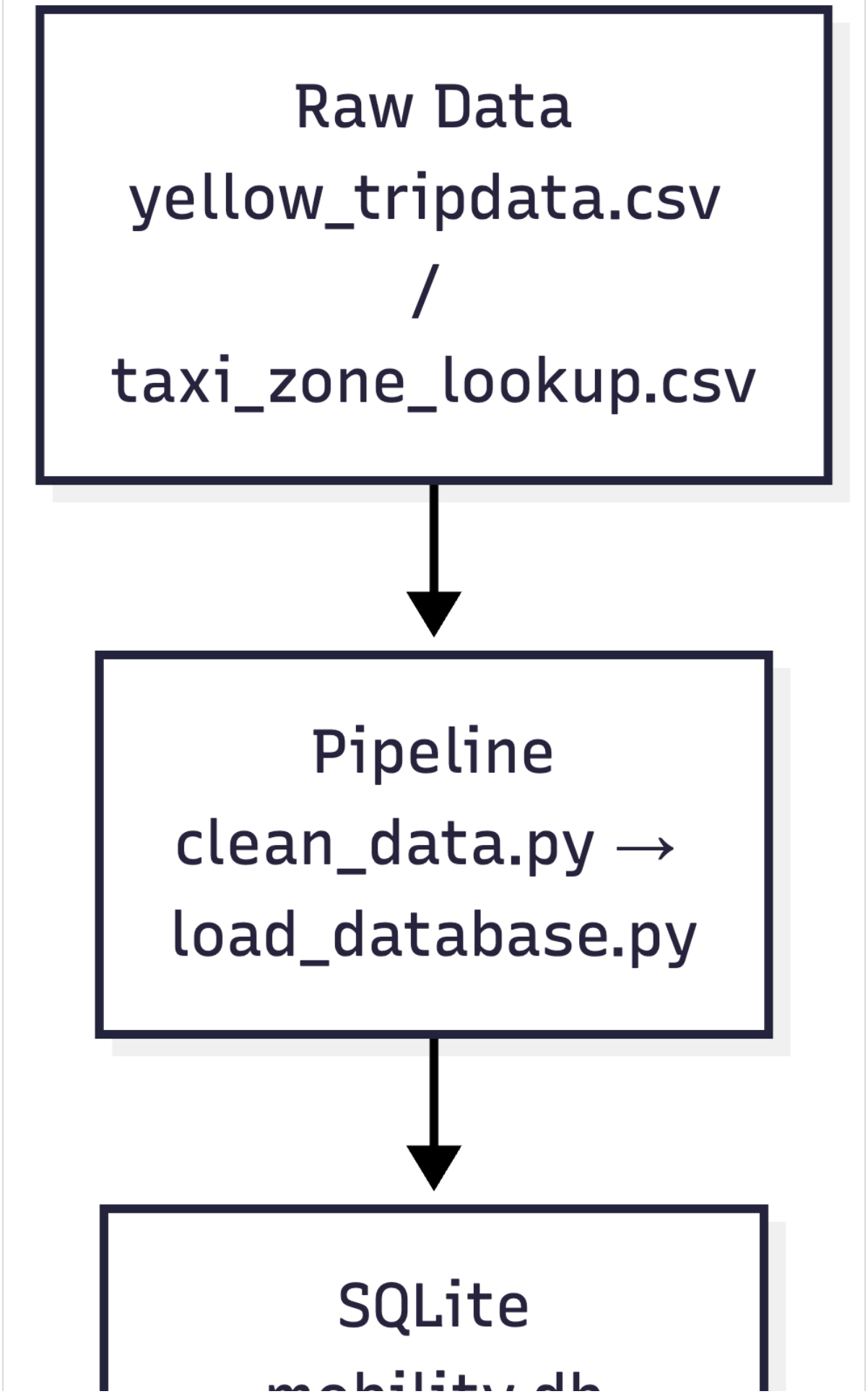
Unexpected Observation

The most surprising finding was the scale of the unknown zone problem. We expected a small number of GPS failures, but 154,335 dropoff zones and 152,317 pickup zones were unresolvable — nearly 2% of all clean trips. This influenced our design decision to keep these trips in the database rather than exclude them, and to display Unknown explicitly in the dashboard. It also led us to filter unknown zones out of the top-routes table by default, because otherwise Unknown to Unknown appears near the top of the rankings — a recording artefact, not a real mobility pattern.

2. System Architecture and Design Decisions

This section was written by Alier Piel, who was responsible for the Flask API, database schema, and query routing logic.

Architecture Diagram


Raw Data
yellow_tripdata.csv
/
taxi_zone_lookup.csv
Pipeline
clean_data.py →
load_database.py
SQLite

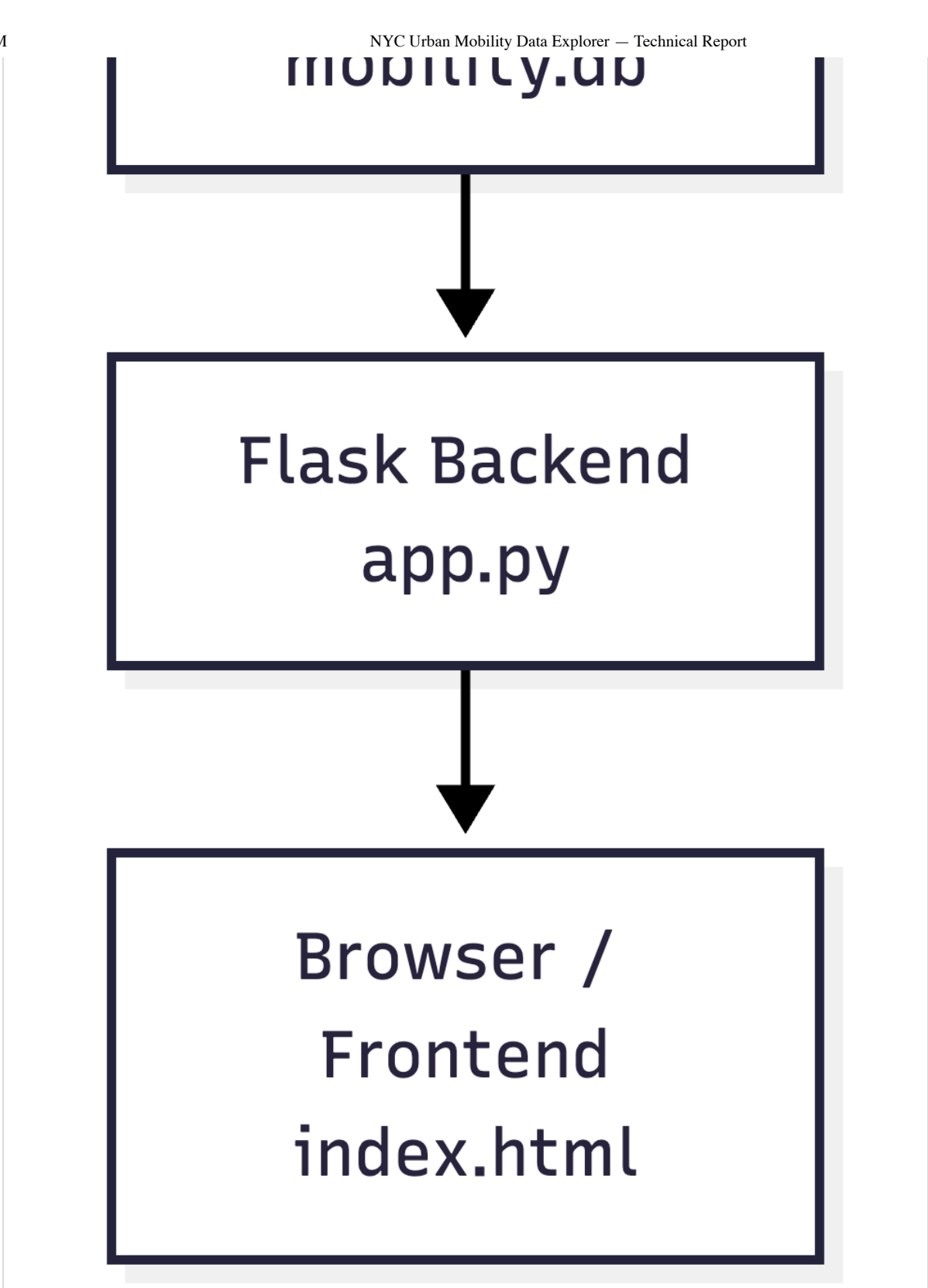


Figure 1 — Three-tier architecture: pipeline → SQLite → Flask API → browser dashboard

System Overview

The application follows a three-tier architecture with a data pipeline feeding the storage layer.

- **Pipeline layer:** `clean_data.py` cleans, normalises and enriches the trip records, producing `trips_clean.csv`, `excluded_records.csv`, `suspicious_records.csv`

and `cleaning_summary.json`. `load_database.py` then creates the schema, inserts data in 250,000-row chunks, builds the two summary tables, then builds all indexes after the bulk load. It finishes with a verify step that checks foreign key integrity and confirms both summary table totals match the trips table exactly.

- **Data layer:** A single SQLite database at `data/mobility.db`, **2.6 GB** in size, stores 7,578,890 cleaned trips plus five dimension tables, two summary tables, two transparency tables and a `dataset_info` table.
- **Backend layer:** A Flask application at `backend/app.py` exposes 12 JSON API endpoints. It also serves the frontend files directly, so the single command `python backend/app.py` starts the entire application on port 5000. Every filter value is passed as a parameterised query placeholder and sort column names come from a fixed whitelist in `db.py`, which prevents SQL injection.
- **Frontend layer:** A browser-based dashboard at `frontend/index.html`, written as vanilla JavaScript ES modules, renders SVG charts, a weekday-by-hour heatmap, a choropleth map, a sortable paginated trip table, and KPI cards — all without any chart, map or frontend framework library.

Stack Choices and Justification

Flask was chosen because the entire team was already working in Python for the pipeline, and Flask's minimal structure was sufficient for a read-only JSON API without requiring a new framework.

SQLite was chosen over PostgreSQL or MySQL because the dataset is one month, built once and then only read. SQLite requires no server process or configuration and ships as a single portable file. The loader turns journaling off during the bulk insert for speed, because a failed build is simply rerun from scratch. The API opens the database in read-only mode using `mode=ro` in the connection URI, so dashboard requests can never modify data.

Vanilla JavaScript with hand-drawn SVG was chosen to avoid a build step and external dependencies, and to demonstrate direct understanding of the browser's rendering model. It also gave full control over the linked hover interaction between the two hourly charts and over the choropleth colour scale.

Schema Structure

The database uses a **star schema** normalised to third normal form (3NF):

- **Fact table** `trips`: one row per cleaned ride, 7,578,890 rows, 29 columns — `trip_id`, `source_row` (links back to the original raw file for traceability), 18 TLC fields, and 9 derived features.
- **Dimension tables:** `zones`, `boroughs`, `vendors`, `rate_codes`, `payment_types` — small lookup tables referenced by foreign key from `trips`. A zone name is stored once, not repeated 7.5 million times.

- **Integrity:** foreign keys plus CHECK constraints repeat the cleaning rules inside the database. For example `trip_distance` must be greater than 0 and at most 100, `avg_speed_mph` must be between 0 and 80, and `dropoff_datetime` must be after `pickup_datetime`. A row that violates any rule cannot be inserted even if the pipeline changes.
- **Summary tables:** `zone_date_hour_stats` with 100,762 rows (one row per date, pickup zone and hour) and `route_date_stats` with 321,988 rows (one row per date, pickup zone and dropoff zone). These store sums and counts rather than averages, so any filtered combination can be re-aggregated correctly.
- **Transparency tables:** `excluded_records` and `exclusion_reasons` — the full audit log of every removed record.

Trade-offs

SQLite vs PostgreSQL: SQLite does not support concurrent writes, but since the dashboard is entirely read-only after the database is built, this was not a constraint. The benefit was zero server configuration and a single portable file.

Pre-aggregated summary tables: Building `zone_date_hour_stats` and `route_date_stats` adds storage and a longer build time, but reduces the most common dashboard queries from scanning 7.5 million rows to scanning around 100,000 summary rows. The API's `source_for()` function uses a summary table only when every active filter is one it can answer. Distance, fare, payment type and airport filters fall back to the full trips table.

Query planning for sorted trips: For a request like the first 25 trips matching a filter sorted by a column, SQLite can either sort every matching row or walk the sort column's index in order. The second approach is much faster when the filter matches many rows — for example all of Manhattan. The API counts matches up to 50,000 rows and forces the sort column's index using `INDEXED BY` when the filter is broad. Without this, a sorted page of Manhattan trips took approximately 30 seconds. With the sort indexes in place it takes approximately 0.2 seconds.

Hand-drawn SVG vs a charting library: Using Chart.js or D3 would have been faster to implement, but writing SVG directly gave full control over layout, colour and interaction, and avoided adding large dependencies to a project that is otherwise dependency-free on the frontend.

3. Algorithmic Logic and Data Structures

This section was written by Chol Mach Kuol, who designed and implemented the custom algorithms in `backend/algorithms.py`.

Three functions in `backend/algorithms.py` are called directly by `app.py` at runtime for ranking, counting and statistics.

3.1 top_k(items, k, key)

Problem: The `/api/top-routes` endpoint ranks 31,440 distinct routes (excluding unknown zones) and returns the top 10. The zone detail panel needs the top 5 of up to 263 destinations. Sorting the full list on every request is wasteful when only the top few are needed.

Approach: A min-heap of fixed size k is maintained. For each item, if its score exceeds the smallest item currently in the heap, it replaces it. At the end, the heap contains exactly the k largest items. The heap operations use Python's heapq library. The algorithm design, the key function and the tie-breaking logic that uses zone IDs to guarantee a consistent result on every run are the team's own work.

```
heap = []
for each item in items:
    val = key(item)
    if size of heap < k:
        push (val, item) onto heap
    else if val > minimum value in heap:
        replace minimum in heap with (val, item)
return heap sorted by val descending
```

Time complexity: O(n log k) time, O(k) space. With n = 31,440 routes and k = 10, this performs approximately 31,440 × 3.3 comparisons against approximately 31,440 × 15 for a full sort.

3.2 count_by(items)

Problem: The zone detail panel needs to count how many trips went to each dropoff zone from a list of trip records fetched for a single pickup zone. Its output feeds `top_k` to find the top 5 destinations.

Approach: A single pass over the list, incrementing a dictionary counter for each value seen. No sorting, no library functions.

```
counts = empty dictionary
for each item in items:
    if item is in counts:
        counts[item] = counts[item] + 1
    else:
        counts[item] = 1
return counts
```

Time complexity: O(n) time, O(u) space where u is the number of unique values — at most 265 zones. Dictionary lookup and insertion are O(1) on average.

3.3 quickselect_median(values)

Problem: Fare distributions are right-skewed — airport flat fares and long trips push the arithmetic mean above what a typical passenger pays. The median is more robust, but sorting the full list costs O(n log n).

Approach: Quickselect — a partition-based selection algorithm derived from quicksort. A pivot is chosen from the middle of the current window. Two pointers move toward each other, swapping elements on the wrong side of the pivot. The algorithm narrows its window to whichever side contains the target index k, using a loop rather than recursion. For an even-length list, the median is the average of the two middle elements, requiring two separate selections. The input list is copied before partitioning so the caller's data is not reordered.

```
quickselect(lst, k):
    lo, hi = 0, length - 1
    while lo < hi:
        pivot = lst[(lo + hi) // 2]
        i, j = lo, hi
        while i <= j:
            advance i while lst[i] < pivot
            retreat j while lst[j] > pivot
            if i <= j: swap lst[i] and lst[j]; i++; j--
        if k <= j: hi = j
        else if k >= i: lo = i
        else: break
    return lst[k]

if n is odd:  median = quickselect(copy, n // 2)
if n is even: median = (quickselect(copy, n//2 - 1) + quickselect(copy,
n//2)) / 2
```

Time complexity: O(n) average time, O(n) space because the input is copied. Worst case is O(n²) if the pivot is always the minimum or maximum, which is rare with a middle-index pivot on real fare data.

4. Insights and Interpretation

All speeds in this section are weighted: total distance divided by total time for the group, not an average of per-trip speeds. This means a one-minute crawl does not count as much as a 40-minute trip, giving a fair picture of how fast the city is actually moving.

Insight 1 — The slower the city, the more each mile costs

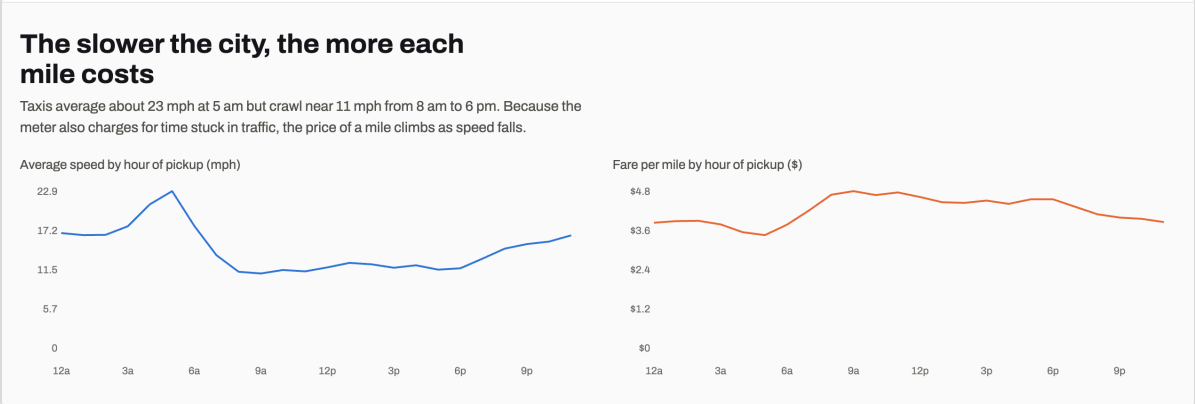


Figure 2 — Average speed (mph) and fare per mile ($) by hour of day across January 2019

The by-hour chart shows that taxis move fastest at 5 am, averaging **22.9 mph**, and slowest at 9 am, averaging **10.9 mph**. Over those same hours, the fare per mile rises from **$3.46 at 5 am to $4.81 at 9 am**. The pattern holds across the entire day: whenever speed falls, fare per mile rises. This is a direct consequence of how the TLC meter works — it charges for both distance travelled and time elapsed, so a mile driven slowly through traffic costs more than the same mile driven at night. A passenger making the same physical journey pays approximately **39% more at 9 am than at 5 am**, purely because of congestion. The relationship is consistent across all 31 days of January 2019, confirming that traffic is the primary driver of fare variability in New York City, not distance.

Insight 2 — Thursday evening is the city's busiest hour; Thursday morning is its slowest

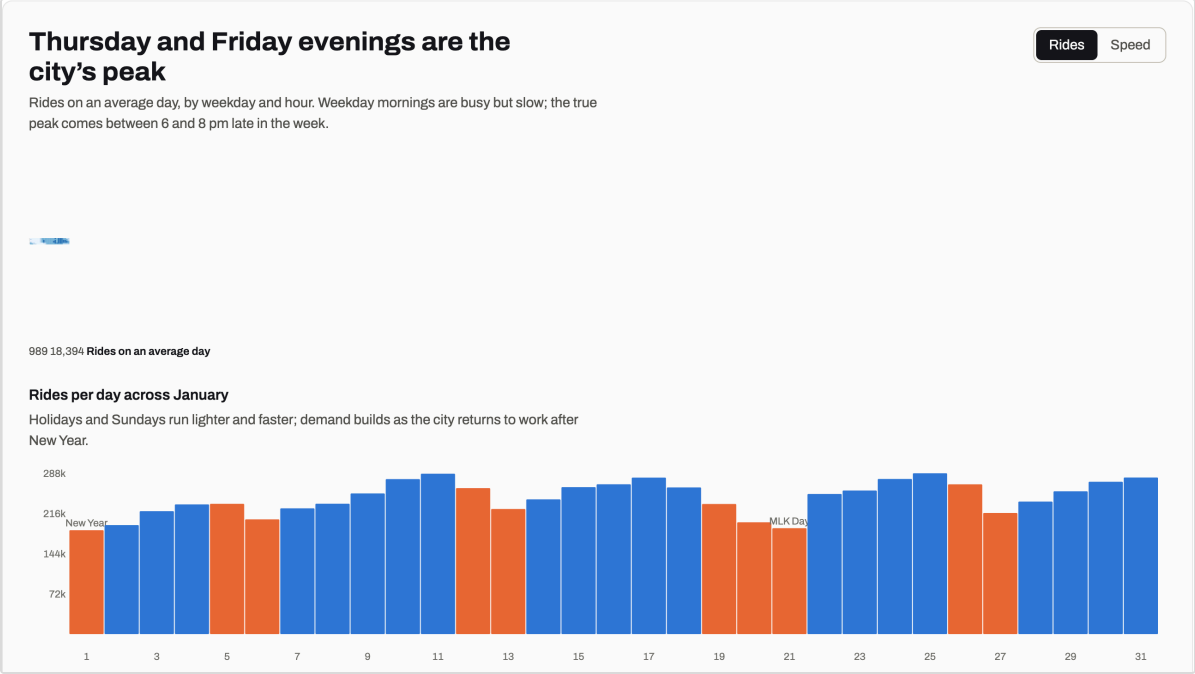
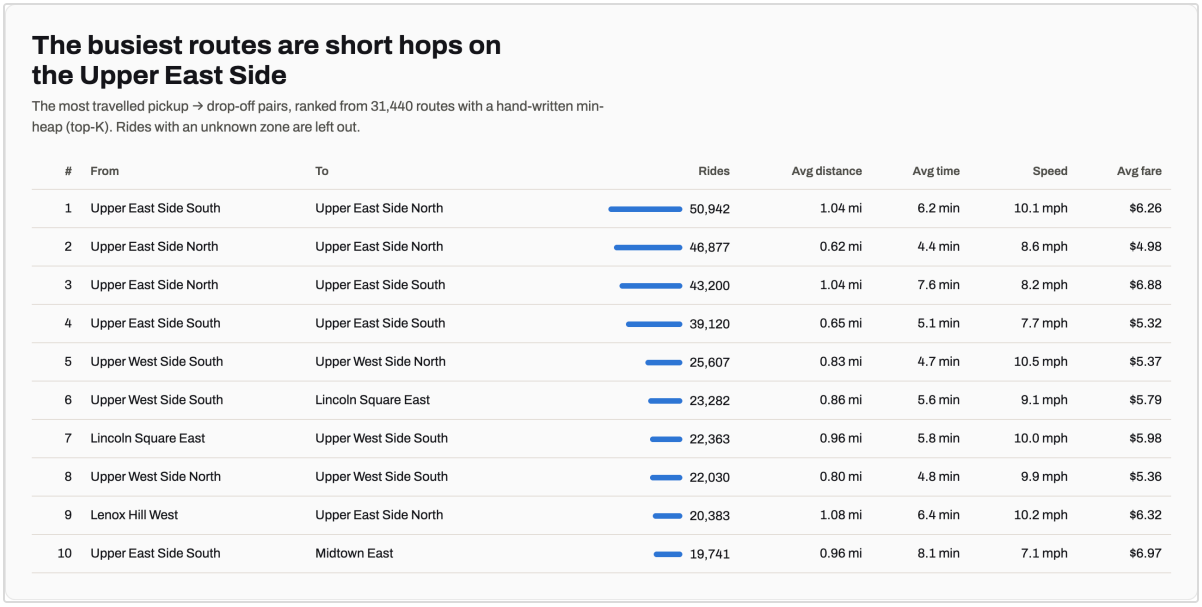


Figure 3 — Trip volume heatmap: weekday (rows) × hour of day (columns), January 2019

The weekday-by-hour heatmap divides trip counts by the number of each weekday in January 2019 so that weekdays with five occurrences are not unfairly compared to those with four. In trip volume, the single busiest cell is **Thursday at 6 pm with 91,737 trips**, followed by Wednesday at 6 pm with 86,905 and Thursday at 7 pm with 85,608. All five of the busiest cells are Thursday or Wednesday evenings. In speed, the slowest cell is **Thursday at 9 am at 9.3 mph**, followed by Thursday at 11 am at 9.7 mph and Thursday at 10 am at 9.8 mph. Thursday is therefore both the busiest evening and the slowest morning of the week. This reveals that peak taxi demand and worst congestion occur on the same day but at different times — the evening surge is driven by social and commuter trips, while the morning slowdown reflects the full weight of business traffic on the same streets.

Insight 3 — 90.8% of rides start in Manhattan, and the busiest routes are short hops



The busiest routes are short hops on the Upper East Side

The most travelled pickup → drop-off pairs, ranked from 31,440 routes with a hand-written min-heap (top-K). Rides with an unknown zone are left out.

#	From	To	Rides	Avg distance	Avg time	Speed	Avg fare
1	Upper East Side South	Upper East Side North	50,942	1.04 mi	6.2 min	10.1 mph	$6.26
2	Upper East Side North	Upper East Side North	46,877	0.62 mi	4.4 min	8.6 mph	$4.98
3	Upper East Side North	Upper East Side South	43,200	1.04 mi	7.6 min	8.2 mph	$6.88
4	Upper East Side South	Upper East Side South	39,120	0.65 mi	5.1 min	7.7 mph	$5.32
5	Upper West Side South	Upper West Side North	25,607	0.83 mi	4.7 min	10.5 mph	$5.37
6	Upper West Side South	Lincoln Square East	23,282	0.86 mi	5.6 min	9.1 mph	$5.79
7	Lincoln Square East	Upper West Side South	22,363	0.96 mi	5.8 min	10.0 mph	$5.98
8	Upper West Side North	Upper West Side South	22,030	0.80 mi	4.8 min	9.9 mph	$5.36
9	Lenox Hill West	Upper East Side North	20,383	1.08 mi	6.4 min	10.2 mph	$6.32
10	Upper East Side South	Midtown East	19,741	0.96 mi	8.1 min	7.1 mph	$6.97

Figure 4 — Choropleth map of pickup density by zone, and top 10 routes by trip count

The choropleth map shows that pickup activity is overwhelmingly concentrated in Manhattan. Of the 7,578,890 clean trips, **6,879,758 — exactly 90.8% — started in Manhattan**. Queens accounts for 5.9%, and every other borough combined accounts for less than 2%. The top routes table confirms the pattern: the single busiest route in January 2019 was **Upper East Side South to Upper East Side North**, with 50,942 trips averaging 1.04 miles and 6.2 minutes. The second busiest was Upper East Side North to Upper East Side North — the same neighbourhood to itself — with 46,877 trips averaging just 0.62 miles and 4.4 minutes. All ten of the busiest routes are short intra-Manhattan trips under 1.1 miles. Yellow taxis in January 2019 functioned primarily as a short-distance urban transit option for Manhattan residents, covering distances that walking, buses or bikes could also serve — a finding with direct implications for congestion pricing policy.

5. Reflection and Future Work

Alier Piel — Backend and Database Schema

This section was written by Alier Piel, who was responsible for the Flask API, database schema, and query routing logic.

The most challenging part of the backend was designing `source_for()` in `app.py`. The pre-aggregated summary tables answer most dashboard queries in milliseconds, but they only support a subset of filters — date range, borough, zone, weekday and hour. Any request that adds a distance, fare, payment type or airport filter must fall back to the full 7.5 million row trips table. Writing the logic to detect which filters are active and automatically choose the right table — without the API caller needing to know — required careful thought about which columns exist in each table and how to avoid returning wrong results when a filter is not supported by the summary table.

The second challenge was query planning for the sorted trip table. SQLite cannot tell the difference between a selective filter covering a few hundred rows and a broad filter covering 6.8 million rows. For broad filters, asking SQLite to sort every match before returning the first 25 took approximately 30 seconds. The solution was to count matches up to 50,000 rows first, and if the filter is broad, force SQLite to use the sort column's index with `INDEXED BY`. This reduced the response time to approximately 0.2 seconds.

Chol Mach Kuol — Algorithms

This section was written by Chol Mach Kuol, who was responsible for all algorithm implementations in `backend/algorithms.py`.

Building `count_by` and `quickselect_median` without standard library functions forced careful thinking about what each function actually does at the level of individual operations. The hardest part was getting quickselect correct for even-length lists. The median of an even-length list requires two separate calls to the partition function with different target indices, and getting the index arithmetic right for both odd and even cases took several iterations of testing against known values. The loop-based implementation rather than a recursive one also required careful reasoning about when the lo and hi pointers have converged and the target element is already in place.

Kuol Akech — Frontend

This section was written by Kuol Akech, who was responsible for the entire frontend dashboard.

The most difficult part was building the choropleth map entirely in SVG without a mapping library. Projecting GeoJSON polygon coordinates onto a fixed SVG canvas required

implementing a manual bounding-box projection — finding the minimum and maximum longitude and latitude across all zone polygons, then scaling each coordinate to fit the canvas dimensions. At New York's latitude, a degree of longitude is shorter than a degree of latitude, so the x axis must be scaled by the cosine of 40.7 degrees. Without this correction the city appears stretched sideways. Zoom using the mouse wheel centred on the cursor and pan using drag are implemented by moving the SVG viewBox. A drag only starts after 4 pixels of movement so that a click on a zone still selects it rather than starting a pan. In `main.js`, every filter change refreshes each dashboard section independently, and older requests are cancelled with `AbortController` so a slow server response can never overwrite a newer one.

Abay Tessema — Pipeline and Data Loader

This section was written by Abay Tessema, who was responsible for the data pipeline, cleaning logic, feature engineering and database loader.

The main challenge was handling the variety of data quality issues without over-removing valid records. The decision of where to draw the line — exclude versus repair versus flag — required reading the TLC data dictionary carefully and making judgment calls about which anomalies were genuine errors versus quirks of how drivers use the system. The 115,074 zero-passenger trips were the clearest example: a naive approach would exclude them, but they are distributed uniformly across all hours and vendors, which strongly suggests a data-entry habit rather than an error. Removing them would have introduced a systematic bias rather than correcting one.

The loader was designed to be safe to rerun from scratch. It drops and recreates all tables, inserts data in 250,000-row chunks to avoid memory pressure, builds the summary tables from the inserted trips, then builds all indexes after the bulk load — because maintaining an index during a 7.5 million row insert is significantly slower than building it once at the end. It finishes with a verify step that checks foreign key integrity and confirms that the summary table totals match the trips table total exactly.

Future Improvements

- **Deployment:** The application could be deployed to AWS using EC2 for the Flask backend and S3 for static frontend assets. The HOST and PORT are already configurable via environment variables in `app.py`, so no code changes would be needed.
- **More months of data:** Adding the full 2019 dataset of approximately 84 million trips would allow seasonal analysis — comparing January's cold-weather patterns against summer and autumn. At that scale the database would need PostgreSQL with date partitioning instead of a single SQLite file.
- **Real-time pipeline:** The TLC publishes trip data with a one-month lag. A scheduled pipeline could automatically fetch and load each new month's data as it becomes

available, keeping the dashboard current without manual intervention.

- **Predictive features:** With a full year of data, a time-series model trained on the zone-by-hour summary table could forecast demand by zone and hour, useful for both passengers planning journeys and drivers planning their working day.

Data source: NYC Taxi & Limousine Commission, Yellow Taxi Trip Records, January 2019; Taxi Zone Lookup and Shapefile. | Built with Flask, SQLite, and hand-drawn SVG.